# 编译器设计专题实验课2026

2026.04

# 一、实验回顾

- 第一次实验，实现一个模拟的**DFA**，包括状态，状态转换等等，输入一串字符，输出可接受的字符流

## 1 实验任务

在当前环境下，编写一个 C/C++ 语言程序，模拟 DFA 运行实现过程。

## 2 实验内容

1. DFA 输入包括“字符集”、“状态集”、“开始状态”、“接受状态集”、“状态转换表”；

2. 上述输入的五元组保存在一个文本文件中；

3. 对 DFA 进行检查，开始状态是否唯一，包含在状态集中，接受状态集是否为空，包含在状态集中；

4. DFA 的输出，输入待显示的字符串的最大长度 N，输出以上定义的 DFA 的语言集中长度 <=N 的所有规则字符串；

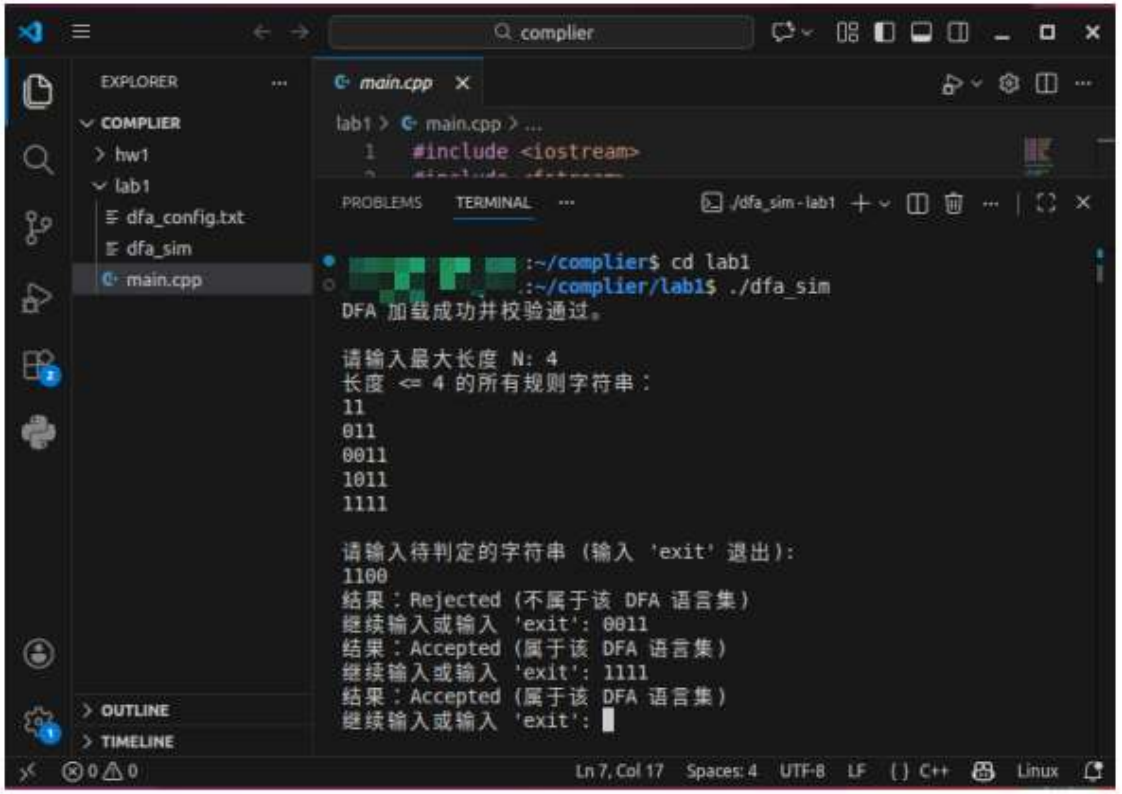
5. DFA 的规则判定，输入（或用字符集随机生成）一个字符串，模拟 DFA 识别字符串的过程判定该字符串是否是规则字符串（属于 DFA 的语言集）；

## 3 实验结果

实验结果如图 1 和图 2 所示

# 一、实验回顾

- 第一次实验，以同学作业为例




本次 DFA 模拟实验让我将抽象的有限自动机五元组理论转化为具体的 C++ 工程实现，在基础功能上，我通过标准模板库的 set 和 map 完成了状态空间与状态转移图的构建，并基于广度优先搜索实现了长度有界语言集的枚举以及特定输入串的判定。

问：本实验需要完成一个能配合教材主文法的编译器，目标是ARM64，大家对于ai代码工具掌握情况怎样，预计花多少时间可以完成此项实验？

答：非常熟悉，1天以内

# 一、实验回顾

- 选做部分

## 编译原理实验平台 (Compilation Web)

这是一个风格明亮、现代化的编译原理实验网页版平台，旨在将传统的编译原理实验（如 DFA 模拟、LL(1) 语法分析、LR(1) 语法分析等）通过可视化的方式在网页端直接展示，提升学习体验、交互性和对算法的理解。

### ✨ 特性

- **现代化 UI**: 基于 HTML5 + Tailwind CSS (CDN) 构建，界面明亮、结构清晰的响应式设计。
- **纯前端架构**: 无需复杂的后端或 Node.js 构建工具，纯浏览器原生运行，克隆即用。
- **动态可视化**: 借助 `vis-network` 图形库，将晦涩难懂的状态转移、语法树等数据结构转化为直观的动态交互图。
- **高可扩展性**: 侧边栏支持多级网页导航，预留了多个实验的插槽，方便后续以模块化的形式添加新实验（如 `ll1.js`, `lr1.js`）。
- **人性化输入**: 支持直接手写、结构化表单输入或通过 `txt` 文件一键上传（自动解析与智能回填表单），极大简化了测试数据的准备。

### 🚀 快速开始

本项目为纯静态前端页面，运行非常简单：

1. 克隆或下载本仓库到本地：

```
git clone https://github.com/your-username/compilation-web.git
```

2. 直接使用任何现代浏览器（Chrome、Edge 等）打开目录下的 `index.html`

# 一、实验回顾

- 选做部分

  在git上创建代码仓库，实现这个项目的可视化和一致性

## 项目结构

```
CompilationWeb/
├── index.html      # 主页面：包含侧边栏导航和所有实验模块的 DOM 结构
├── style.css       # 样式文件：页面切换动画与定制 UI 细节
├── app.js          # 核心交互：控制多级网页的切换与通用 UI 逻辑
├── dfa.js          # 实验一模块：DFA 核心类（解析、校验、生成字符串）及
├── dfa.txt         # 测试用例：标准的 DFA 状态转移五元组定义样例
└── .trae/          # AI 辅助开发目录：存放项目专属开发规范 Skill
```

## 当前与后续开发计划

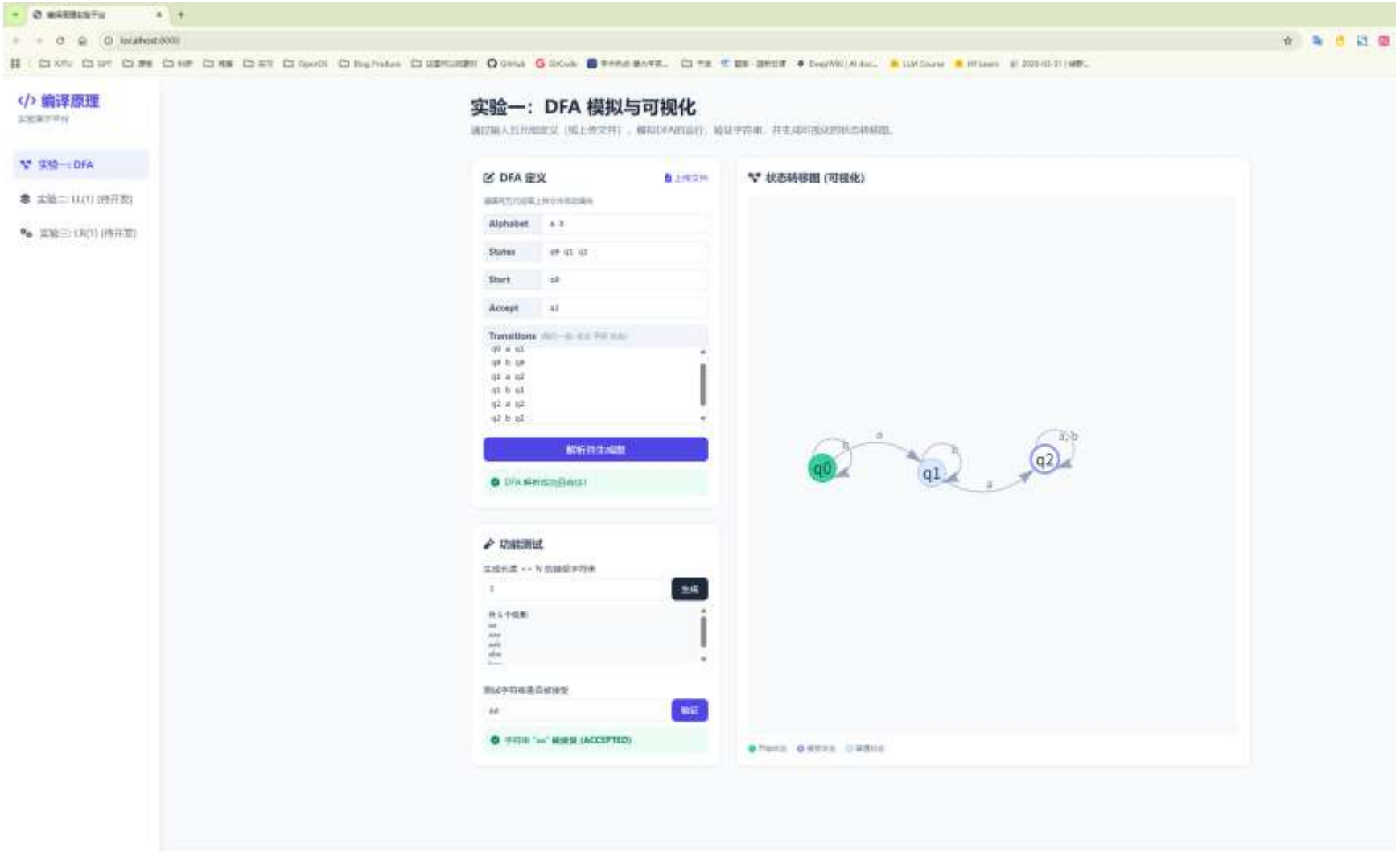
- [x] **实验一：DFA 模拟与可视化**
  - 五元组解析与合法性验证
  - 输入字符串匹配
  - 生成指定长度内的所有接受字符串
  - 自动渲染状态转移图
- [ ] **实验二：LL(1) 语法分析** (规划中)
  - 求解 FIRST 和 FOLLOW 集合
  - 构造预测分析表
  - 字符串分析过程展示
- [ ] **实验三：LR(1) 语法分析** (规划中)

# 一、实验回顾

- 选做部分


编译原理
实验一：DFA
实验二：LL(1) (待开发)
实验三：LR(1) (待开发)
实验一：DFA 模拟与可视化
DFA 定义
上传文件
Alphabet
a b
States
q0 q1 q2
Start
q0
Accept
q2
Transitions
q0 a q1
q1 b q2
q2 a q1
解析并生成图
状态转移图 (可视化)
功能测试
生成
验证

# 一、实验回顾


编译原理
实验一：DFA 模拟与可视化
DFA 定义
Alphabet
States
Start
Accept
Transitions
解析并生成图
状态转移图 (可视化)
q0
q1
q2
功能测试
生成
验证

# 二、实验过程

- 实验一和实验二的关联

dfa_in.dfa.该文件是实验一的输入，根据dfa表输出可接受字符流，该文件在实验二中加载作为词法分析的规则文件

```
root@dsw-544537-7cf95f6b75-g58n9:/mnt/workspace# g++ -o DFAnew DFAnew.cpp
root@dsw-544537-7cf95f6b75-g58n9:/mnt/workspace# ./DFAnew
DFA saved to dfa_in.dfa
root@dsw-544537-7cf95f6b75-g58n9:/mnt/workspace# cat dfa_in.dfa
{
    "states": [
        0,
        1,
        2,
        3
    ],
    "transitions": {
        "0": {
            "*": 3,
            "+": 3,
            "-": 3,
            "/": 3,
            "0": 2,
            "a": 1
        },
        "1": {
            "0": 1,
            "a": 1
        },
        "2": {
            "0": 2
        },
        "3": {}
    },
    "accept_states": {
        "1": "ID",
        "2": "NUM",
        "3": "OPERATOR"
    }
}root@dsw-544537-7cf95f6b75-g58n9:/mnt/workspace#
```

# 二、实验过程

- 实验2

```
root@dsw-544537-7cf95f6b75-g58n9:/mnt/workspace# g++ scanner.cpp -o scanner
root@dsw-544537-7cf95f6b75-g58n9:/mnt/workspace#
```

```
hostname: you must be root to change the host name
root@dsw-544537-7cf95f6b75-g58n9:/mnt/workspace# ./scanner test.src
Error: Invalid character 'x' at 1:1
Error: Invalid character ' ' at 1:2
Error: Invalid character '=' at 1:3
Error: Invalid character '4' at 1:4
Error: Invalid character '2' at 1:5
Error: Invalid character ' ' at 1:6
Error: Invalid DFA state 3
<OPERATOR, "+", 1:7>
root@dsw-544537-7cf95f6b75-g58n9:/mnt/workspace#
```

test.src

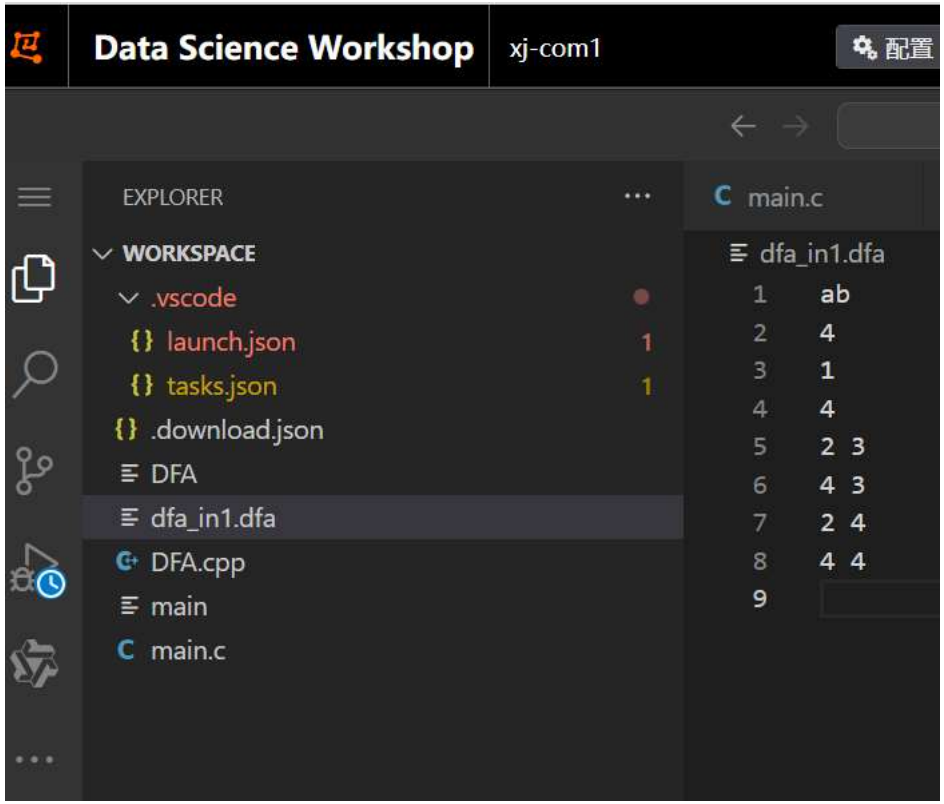
```
1    x =42 +y
```

**当采用第一个dfa作为词法规则时，分析test.src报错，请大家分析报错原因，作为实验三回答问题部分**

扩展DFA如下页所示，可接受ID，NUM和OPERATOR，这个DFA实现了四则运算。

# 二、实验过程

- 实验2




扩展DFA，

```
dfa_in.dfa
{
    "states": [
        0,
        1,
        2,
        3
    ],
    "transitions": {
        "0": {
            "*": 3,
            "+": 3,
            "-": 3,
            "/": 3,
            "0": 2,
            "A": 1,
            "B": 1,
            "C": 1,
            "D": 1,
            "E": 1,
            "F": 1,
            "G": 1,
            "H": 1,
            "I": 1,
            "J": 1,
            "K": 1,
            "L": 1,
            "M": 1,
            "N": 1,
            "O": 1,
            "P": 1,
            "Q": 1,
            "R": 1,
            "S": 1,
            "T": 1,
            "U": 1,
            "V": 1,
            "W": 1,
            "X": 1,
            "Y": 1,
```

回顾检查第一次实验的DFA，DFA的写法不固定，目的是实现DFA的五元组

# 二、实验过程

- 实验一和实验二

1：如何体现自己的标志，在web ide中

```
root@dsw-544537-7cf95f6b75-g58n9:/mnt/workspace# bash
root@dsw-544537-7cf95f6b75-g58n9:/mnt/workspace# export PS1="[\u@\daihuijun \W]\$ "  # 示例格式：显示用户、主机名和当前目录
[root@三 4月 23aihuijun workspace]$
```

在PAI的Terminal中直接修改hostname属于系统级操作，根据知识库内容显示以下关键限制和替代方案：

1. **系统限制**

   PAI-DSW实例的HostName字段是系统自动分配的宿主机名称（如 `smart-scene-cls-854dbdc99d-****` ），属于平台管理的资源标识。**普通用户无权限直接通过Terminal修改宿主机名**，强行修改可能导致实例网络异常或服务不可用。

2. **替代解决方案**

试一下修改PS1变量，是一种临时修改变量方法，需要每次实验之前完成修改，报告贴截图的地方核对实验结果、ps1和报告提交人姓名

Bash

**export PS1="[\u@\daihuijun \W]\$ "  # 示例格式：显示用户、主机名和当前目录**

# 二、实验过程

- 实验2内容

  修改DFA文件，运行时候可以解析任何一个源程序文件，使其支持主文法，在后续验收可以通过随机编译测试。

  可以将输入的json换成其他格式，比如text，csv，正则表达式等

- 实验2报告提交

  修改ps1，在终端、实例名称或者其他地方显示，必做部分请提现一下，

- AI赋能带来了更多更快的编程体验，因此评价的指标每年都在不断变化。

# 三、实验三任务

- 实验3

语法分析则以词法分析输出的单词序列为输入，依据语言的语法规则，构建出对应的语法分析表。

构建给定文法的LR(0)项目集规范族（Canonical Collection of LR(0) Items），为后续生成SLR(1)分析表提供基础。

实验要求

**输入文法规则，支持扩展巴科斯范式（如 E → E +T | T）。**
**需要手动增广文法：添加新的起始产生式 S' → S（S为原文法起始符号）。**
**输出：生成LR(0)项目集规范族，**计算每个状态的闭包（Closure）和状态转移（Goto）。
输出所有项目集、如可以实现规范族的图形化表示（如状态编号和转移关系）。需要考虑是否属于LR(0)文法（无移进-归约或归约-归约冲突）

# 三、实验三任务

- 实验3步骤

1.增广文法
添加新的起始产生式 S' → S，确保文法有唯一接受状态。

2.生成初始项目集
从增广文法的初始产生式 S' → ·S 开始，构建初始项目集 I0。

3.计算闭包（Closure）
对每个项目集，递归添加所有可达的LR(0)项（如 A → α·Bβ 需添加 B → ·γ）。

# 三、实验三任务

- 实验3

4. 计算转移(Goto)

对每个项目集和文法符号，计算转移后的新项目集。

5. 结果生成

重复步骤3-4，直到不再产生新项目集。

例如

$S' \rightarrow E$

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid id$

$\check{E} \rightarrow E \mid \check{E}, E$

$E \rightarrow i \mid d \mid d[\check{E}] \mid d(\check{E}) \mid E + E \mid E * E$

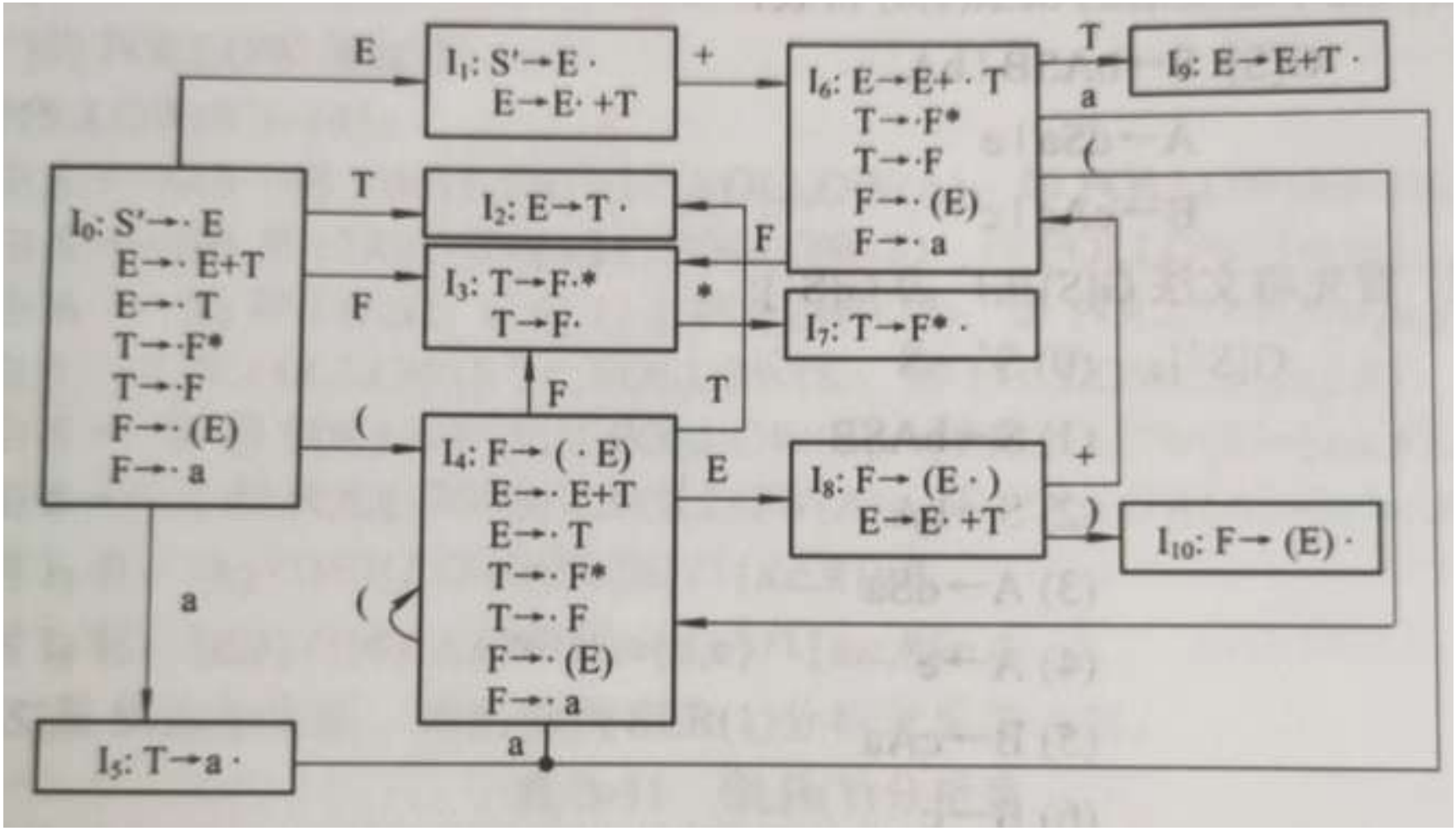
$B \rightarrow E\ r\ E \mid E$

(1) G'

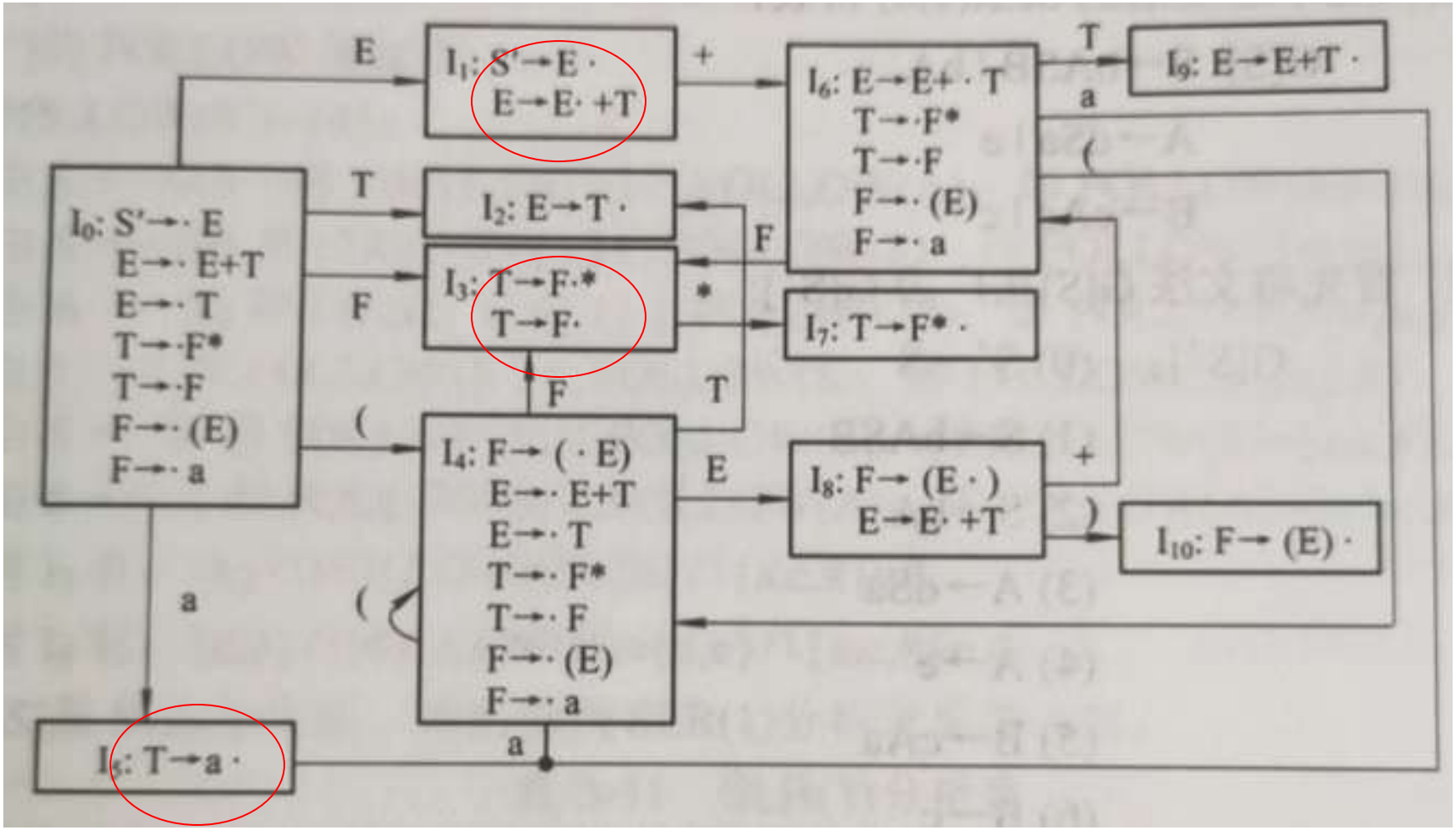
| | |
|---|---|
| E'->E | (1) |
| E->E+T | (2) |
| E->T | (3) |
| T->T*F\|F | (4) |
| T->F | (5) |
| F->(E) | (6) |
| F->-F | (7) |
| F->id | (8) |

# 三、实验三任务

- LR (0) 项目集（Item Set） 及其转移关系



# 三、实验三任务




多个状态里同时存在归约和移进项目，或者多个归约项目，违反了 LR (0) 项目集的定义，转移逻辑错误 $I_0$ 到 $I_3$ 的转移、$I_6$ 到 $I_7$ 的转移，都没有正确处理所有项目；项目闭包不完整；图里没有体现任何移进 / 归约冲突

# 四、作业讲解

习题 1.3□图 1-8 中是一个滚木雷石玩具，在 $A$ 或 $B$ 垛口处扔下一个木球，机关 $x_1$、$x_2$ 和 $x_3$ 让木球落向左方或右方，每当一个木球遇到一个机关时，就引起这个机关在木球通过之后改变方向，所以下一个木球会走相反的暗道。

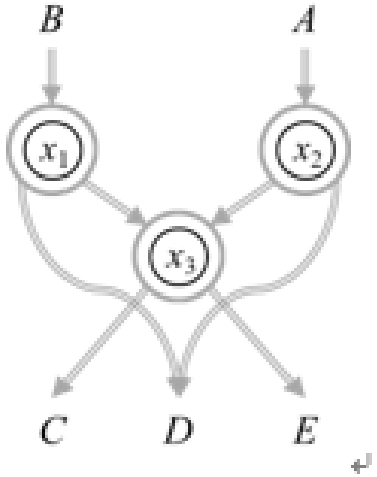



图 1-8□滚木雷石玩具

(1)|用有穷自动机为这个玩具建模。设输入为从垛口 $A$ 或 $B$ 扔进一个木球。设接受对应于木球从 $D$ 或 $E$ 垛口出来，不接受则表示木球从 $C$ 垛口出来。

(2)|用自然语言描述这个自动机的语言。

# 四、作业讲解

**(1) 有限自动机建模**

**状态集**

用 $a,b,c$ 分别表示机关 $x_1,x_2,x_3$ 的方向，0 = 左，1 = 右。

用 $d$ 表示最近一次投球的出口，$d \in \{C,D,E,\epsilon\}$（$\epsilon$ 表示尚未投球）

状态：

$$Q = \{q_{abc}^{d} \mid a,b,c \in \{0,1\}, d \in \{C,D,E,\epsilon\}\}$$

共 $8 \times 4 = 32$ 个状态。

**初始状态**

$$q_0 = q_{000}^{\epsilon}$$

**输入字母表**

$$\Sigma = \{A,B\}$$

**转移函数** $\delta(q_{abc}^{d},X) = q_{a'b'c'}^{o}$，其中：

**接受状态集**

$$F = \{q_{abc}^{d} \mid d \in \{D,E\}\}$$

**(2) 自然语言描述**

该自动机的语言是所有由 $A$ 和 $B$ 组成的投球序列，使得在初始机关全左的情况下，按照规则依次投球（每次投球后所经过的机关翻转方向），序列中最后一个木球从出口 $D$ 或 $E$ 掉出，而不是从出口 $C$ 掉出。

转移函数表示为$\delta\big((x_1, x_2, x_3), A\big)=\big((x_1, \overline{x_2}, \overline{x_3}), (x_1, \overline{x_2}, x_3)\big)$

$\delta\big((x_1, x_2, x_3), B\big)=\big((\overline{x_1}, x_2, \overline{x_3}), (\overline{x_1}, x_2, x_3)\big)$

一共是16个状态转移函数

不管是多少种状态，16/32，转移函数和状态数要相一致。

# 四、作业讲解

习题 1.5□分别写出下列语言的字母表并依次判断符号串 ε、123.、+5e6 是不是那个字母表上的符号串。

(1)|无符号八进制定点数；

(2)|有符号十进制定点数；

(3)|无符号十进制实数。

**定点数是有小数点的数**

- **(1) 无符号八进制定点数**
  - **字母表:** {0, 1, 2, 3, 4, 5, 6, 7}
  - **判断:** ε (是), 123 (是), +5e6 (否)
- **(2) 有符号十进制定点数**
  - **字母表:** {0, 1, . . . , 9, + , − }
  - **判断:** ε (是), 123 (是), +5e6 (否)
- **(3) 无符号十进制实数**
  - **字母表:** {0, 1, . . . , 9, . , e, E}
  - **判断:** ε (是), 123 (是), +5e6 (否)

# 四、作业讲解

习题 1.7□写出字母表{+,·0,·1}上同时满足如下条件的语言 *B*:

(1)串的长度不超过 4;

(2)串除以 5 余数为 0;

(3)同时满足(1)和(2)的串一定属于 *B*。

B，完全枚举加判断。

0，00，000，101，　　0000，0101，　1010，　1111

+0，+00，+000，+101，　　~~+0000，+0101，+1010　　+1111~~

一共 12 个。

这个只是针对二进制，

# 四、作业讲解

习题 2.1□分别写出字母表{0,·1}上下列语言的 DFA：

(1)|有符号的二进制整数，不含前 0。

(2)|无符号二进制定点数，不含后 0。

(3)|带有偶数个 0 做子串的串的集合。

(4)|串中 0 的个数是 3 的串的集合。

$Q = \{q_0, q_1, q_2, q_3, q_4, q_5, q_d\}$

$\Sigma = \{0,1,.\}$

$q_0$ 为初始状态

$F = \{q_1, q_2, q_4\}$

| 当前状态 | 输入 0 | 输入 1 | 输入 . |
|---|---|---|---|
| $q_0$ | $q_2$ | $q_1$ | $q_3$ |
| $q_1$ | $q_1$ | $q_1$ | $q_3$ |
| $q_2$ | $q_2$ | $q_1$ | $q_3$ |
| $q_3$ | $q_5$ | $q_4$ | $q_d$ |
| $q_4$ | $q_5$ | $q_4$ | $q_d$ |
| $q_5$ | $q_5$ | $q_4$ | $q_d$ |
| $q_d$ | $q_d$ | $q_d$ | $q_d$ |

**此次老师课堂更正字母表为{0, 1, .}此处贴出的答案不是正确答案，是较为普遍的答案**

# 四、作业讲解

习题 2.3□设 $A$ 是一个 DFA，$a$ 是 $A$ 的一个输入符号，使得对于 $A$ 的所有状态 $q$，有 $\upsilon(q,a)=q$。

(1)|通过对 $n$ 进行归纳，证明：对所有 $n\geqslant 0$，$\tilde{v}(q,a^n)=q$，其中 $a^n$ 是由 $n$ 个 $a$ 组成的串。

(2)|证明：要么 $\{a\}^*\subseteq L(A)$，要么 $\{a\}^*\cap L(A)=\emptyset$。

**此题几乎所有同学都答对了**

习题 2.6□考虑下列 $\varepsilon$-NFA：

| | $a$ | $b$ | $c$ | $\varepsilon$ |
|---|---|---|---|---|
| $\rightarrow q$ | {} | {$q$} | {$p$} | {$r$} |
| $p$ | {$q$} | {$p$} | {$r$} | {} |
| $*r$ | {$p$} | {$r$} | {} | {$q$} |

(1)|计算每个状态的 $\varepsilon$ 闭包。

(2)|给出这个自动机所接受的长度不大于 3 的串。

(3)|把这个自动机转换为 DFA。

# 四、作业讲解

- 习题 2.6 ε-NFA 定义

| State | a | b | c | ε |
|---|---|---|---|---|
| →q | {q} | {p} | {r} | {} |
| p | {q} | {p} | {r} | {} |
| *r | {p} | {r} | {} | {q} |

$w(q)=\{q,r\}$; $w(p)=\{p\}$; $w(r)=\{q,r\}$

**第二问是20个串**

# 四、作业讲解

习题 2.7□已知 NFA 如转移表所示：

| | 0 | 1 | ε |
|---|---|---|---|
| →1 | | | {2,6} |
| 2 | {3} | | {1} |
| 3 | | | {4} |
| 4 | | {5} | |
| 5 | | | {2,8} |
| 6 | {7} | | |
| 7 | | | {8} |
| *8 | | | |

(1)|计算状态 5 的 ε 闭包。

(2)|用自然语言描述这个自动机所识别的语言。

(3)|把这个自动机转换为 DFA。

# 四、作业讲解

- 闭包是{1，2，5，6，8}
- DFA状态缩减为三种状态